

Error Type Distributions Differ Between RL-Aligned and DPO-Aligned Code Generation Models

Anonymous Authors¹

Abstract

This study examines whether code generation models aligned with different training objectives produce different error type distributions among failed generations. Two models were compared: CodeRL (770M parameters, trained with execution-based reinforcement learning) and CodeLlama-7B-Instruct (7B parameters, instruction-tuned). On a combined dataset of HumanEval+ and MBPP+ benchmarks (542 problems, 766 total failures), error type distributions differed between the two models (chi-square = 35.27, $p = 2.19 \times 10^{-8}$, Cramér’s $V = 0.21$). Among RL model failures, 2.12% were assertion errors compared to 0% for the instruction-tuned model (Fisher’s exact test, $p = 0.0027$). Using execution tracing, RL-generated code executed a mean of 29.4% of lines before failure compared to 0.09% for the instruction-tuned model (Welch’s t-test, $p = 1.08 \times 10^{-34}$, Cohen’s $d = 1.69$). The effect size increased from Cramér’s $V = 0.21$ at the coarse 3-category taxonomy to $V = 0.82$ at the fine-grained 19-category LlmFix taxonomy. These results are confounded by differences in model architecture (encoder-decoder vs. decoder-only), model size (770M vs. 7B parameters), and pre-training data. The findings do not establish that alignment method is the causal factor for the observed differences.

1. Introduction

Code generation models fail on substantial portions of standard benchmarks. When evaluating model perfor-

mance, most studies report aggregate pass rates (e.g., pass@k metrics) without examining the composition of failures. Different training objectives may create different optimization pressures that manifest as different failure patterns, but this possibility has not been systematically examined.

This study investigates whether two models trained with different objectives—execution-based reinforcement learning (RL) and instruction tuning—produce different distributions of error types among their failed generations. The comparison uses standard error categories: syntax errors (code fails to parse), runtime errors (code parses but crashes during execution), and assertion errors (code executes completely but produces incorrect output).

The study tests four hypotheses:

1. Error type distributions differ between the two models (existence test)
2. The RL model produces a higher proportion of assertion errors among failures
3. The RL model produces code that executes deeper before failing
4. The effect persists at fine-grained error taxonomy levels

The models compared differ in multiple ways beyond their training objectives, including architecture, parameter count, and pre-training data. These confounds limit causal interpretation. The results describe observed differences between these specific models rather than establishing general properties of alignment methods.

2. Related Work

2.0.1. 2.1 ALIGNMENT METHODS FOR LANGUAGE MODELS

Reinforcement learning from human feedback (RLHF) trains language models using reward signals derived

¹Anonymous Institution, Anonymous City, Anonymous Region, Anonymous Country. Correspondence to: Anonymous Author <anonymous@anonymous.org>

Preliminary work. Under review by the International Conference on Machine Learning (ICML). Do not distribute.

from human preferences (Ouyang et al., 2022). Direct Preference Optimization (DPO) provides an alternative formulation that directly optimizes a preference objective without requiring a separate reward model (Rafailov et al., 2023). Xu et al. (2024) compared PPO and DPO for code generation, reporting that PPO achieved higher pass rates in code competition settings.

2.0.2. 2.2 REINFORCEMENT LEARNING FOR CODE GENERATION

CodeRL (Le et al., 2022) trains code generation models using binary execution rewards, where code that passes test cases receives reward 1 and code that fails receives reward 0. This creates a reward structure where all non-passing programs—whether they fail to parse, crash at runtime, or produce wrong output—receive identical zero reward. Subsequent work has explored variations including curriculum learning with compiler feedback (Dou et al., 2024) and end-to-end execution feedback (Liu et al., 2024b).

2.0.3. 2.3 ERROR TAXONOMY IN CODE GENERATION

Prior work has developed taxonomies for classifying code generation errors. The ICSE 2025 study by Wang et al. (2025) provides a three-tier classification based on execution semantics. The LlmFix framework (Zhang et al., 2024) extends this to 19 distinct error causes. These taxonomies have been used to characterize errors without stratifying by training method.

3. Methodology

3.0.1. 3.1 MODELS

Two models were compared:

CodeRL (RL-aligned): CodeT5-large (770M parameters) with execution-based RL training using binary pass/fail rewards (Salesforce/codet5-large-ntp-py). This model uses an encoder-decoder architecture.

CodeLlama-7B-Instruct (instruction-tuned): CodeLlama with instruction tuning (7B parameters, codellama/CodeLlama-7b-Instruct-hf). This model uses a decoder-only architecture. It is a general instruction-following model, not specifically trained with code-focused DPO.

The models differ in architecture (encoder-decoder vs. decoder-only), parameter count (770M vs. 7B), base pre-training data, and fine-tuning procedure. These differences confound attribution of any observed effects to specific training objectives.

3.0.2. 3.2 DATASETS

Evaluation used HumanEval+ (164 problems) and MBPP+ (378 problems), totaling 542 problems. These benchmarks extend the original HumanEval and MBPP with additional test cases. One sample was generated per problem per model at temperature 0.8, yielding 1,084 total samples.

3.0.3. 3.3 ERROR CLASSIFICATION

Errors were classified into three coarse categories based on Python exception types:

- **Syntax errors:** Code fails to parse (SyntaxError, IndentationError)
- **Runtime errors:** Code parses but raises an exception during execution (TypeError, NameError, ValueError, etc.)
- **Assertion errors:** Code executes completely but produces incorrect output (AssertionError from test cases)

For fine-grained analysis, the LlmFix 19-category taxonomy was applied, distinguishing specific error causes (e.g., indentation_error vs. syntax_error within the syntax category).

3.0.4. 3.4 EXECUTION DEPTH MEASUREMENT

Execution depth was measured using Python’s sys.settrace() function. For each failed sample, the number of unique lines executed before failure was counted and divided by the total number of executable lines in the generated code plus test harness. This provides a measure of how far into the code execution proceeded before failure.

3.0.5. 3.5 STATISTICAL ANALYSIS

- **H-E1 (existence):** Chi-square test for independence on the 2×3 contingency table (model $\times$ error type), with Cramér’s V for effect size
- **H-M1 (assertion proportion):** Fisher’s exact test (one-sided) on the 2×2 table (model $\times$ assertion/non-assertion)
- **H-M2 (execution depth):** Welch’s t-test (unequal variances assumed) with Cohen’s d for effect size
- **H-M3 (fine-grained taxonomy):** Chi-square test with Cramér’s V at both coarse (3-category) and fine (19-category) granularities

4. Experimental Setup

Model	Architecture	Parameters	Training
CodeRL	CodeT5 (encoder-decoder)	770M	Execution RL
CodeLlama-Instruct	Llama 2 (decoder-only)	7B	Instruction tuning

Dataset	Problems
HumanEval+	164
MBPP+	378
Total	**542**

Generation parameters: Temperature = 0.8, n = 1 sample per problem, seed = 42

Execution environment: NVIDIA H100 GPU, PyTorch 2.7.1, Transformers 4.45.0

5. Results

5.0.1. 5.1 PASS RATES

Model	Pass	Fail	Pass Rate
CodeRL	306	236	56.5%
CodeLlama-Instruct	12	530	2.2%

The CodeRL model achieved substantially higher pass rates than CodeLlama-Instruct on these benchmarks. The low pass rate of CodeLlama-Instruct (2.2%) indicates that this general instruction-following model performs poorly on these code generation tasks without additional code-specific fine-tuning.

5.0.2. 5.2 H-E1: ERROR TYPE DISTRIBUTION COMPARISON

Contingency table (failure counts):

Statistical results:

- Chi-square = 35.27, df = 2
- $p = 2.19 \times 10^{-8}$
- Cramér's V = 0.215

The null hypothesis that error type distributions are independent of model is rejected at $p < 0.001$. The effect size ($V = 0.215$) is in the small-to-medium range by conventional standards.

Proportions among failures:

5.0.3. 5.3 H-M1: ASSERTION ERROR PROPORTION

2×2 contingency table:

Statistical results:

- Fisher's exact test (one-sided, greater): $p = 0.0027$
- CodeRL assertion proportion: 2.12% (5/236)
- CodeLlama-Instruct assertion proportion: 0.00% (0/530)
- Odds ratio: undefined (infinite, due to zero count)

The CodeRL model produced a non-zero proportion of assertion errors (2.12%), while CodeLlama-Instruct produced zero assertion errors among 530 failures.

5.0.4. 5.4 H-M2: EXECUTION DEPTH

Descriptive statistics:

Statistical results:

- Welch's t-test: $t = 14.47$, $p = 1.08 \times 10^{-34}$
- Cohen's d = 1.69

The CodeRL model's failures executed substantially deeper into the code (mean 29.4% of lines) compared to CodeLlama-Instruct failures (mean 0.09% of lines). The ratio of means is approximately 326:1. The effect size ($d = 1.69$) is large by conventional standards.

95% Confidence intervals:

- CodeRL: [0.254, 0.334]
- CodeLlama-Instruct: [-0.001, 0.003]

Trace success rate was 100% for both models.

5.0.5. 5.5 H-M3: FINE-GRAINED TAXONOMY

Coarse level (3 categories):

- Chi-square = 33.79, df = 2
- $p = 4.61 \times 10^{-8}$
- Cramér's V = 0.210

Fine-grained level (9 observed categories of 19 possible):

- Chi-square = 525.40, df = 8

	Syntax	Runtime	Assertion	Total
CodeRL	218	12	5	236
CodeLlama-Instruct	529	1	0	530

Error Type	CodeRL	CodeLlama-Instruct
Syntax	92.4%	99.8%
Runtime	5.1%	0.2%
Assertion	2.1%	0.0%

	Assertion	Non-Assertion	Total
CodeRL	5	231	236
CodeLlama-Instruct	0	530	530

- $p = 2.49 \times 10^{-108}$
- Cramér’s $V = 0.823$

The effect size increased from $V = 0.21$ at the coarse level to $V = 0.82$ at the fine-grained level.

Fine-grained distribution:

CodeLlama-Instruct failures concentrated almost exclusively in a single error cause (syntax_error: 99.8%), while CodeRL failures distributed across multiple causes, with indentation_error (69.5%) and syntax_error (22.9%) most common.

6. Discussion

6.0.1. 6.1 SUMMARY OF FINDINGS

The two models produced different error type distributions among their failed generations. The CodeRL model showed:

- A non-zero proportion of assertion errors (2.12% vs. 0%)
- Greater mean execution depth before failure (29.4% vs. 0.09%)
- More distributed errors across fine-grained categories

These differences were statistically significant with effect sizes ranging from small-medium ($V = 0.21$ for coarse error types) to large ($d = 1.69$ for execution depth; $V = 0.82$ for fine-grained taxonomy).

6.0.2. 6.2 INTERPRETATION LIMITATIONS

The observed differences cannot be attributed solely to alignment method differences. The models differ in:

1. **Architecture:** CodeRL uses an encoder-decoder architecture (CodeT5), while CodeLlama-Instruct uses a decoder-only architecture (Llama 2). Architectural differences may affect code generation patterns independent of training objectives.
1. **Model size:** CodeRL has 770M parameters; CodeLlama-Instruct has 7B parameters. Despite having fewer parameters, CodeRL achieved much higher pass rates (56.5% vs. 2.2%), suggesting that model size is not the primary factor in these results.
1. **Pre-training data:** CodeT5 was pre-trained primarily on code, while Llama 2 was pre-trained primarily on general text. This may affect code generation quality independent of fine-tuning.
1. **Fine-tuning procedure:** CodeRL received execution-based RL training specifically for code generation. CodeLlama-Instruct received general instruction tuning, not code-specialized DPO. A fairer comparison would use a model explicitly trained with code-focused DPO.
1. **Base model performance:** The large gap in pass rates (56.5% vs. 2.2%) suggests the models differ substantially in overall code generation capability, which may confound error type comparisons.

6.0.3. 6.3 WHAT THESE RESULTS DO AND DO NOT SHOW

These results show: Two specific models—CodeRL and CodeLlama-7B-Instruct—produce different error type distributions on HumanEval+ and MBPP+. The differences are statistically significant and the effect sizes are non-trivial.

These results do not show: That execution-based RL training causes different error distributions than

Model	Mean Depth	SD	Median	Min	Max	N
CodeRL	0.294	0.311	0.211	0.0	1.0	236
CodeLlama-Instruct	0.001	0.022	0.0	0.0	0.5	530

Error Cause	CodeRL (n=236)	CodeLlama-Instruct (n=530)
indentation_error	164 (69.5%)	0 (0.0%)
syntax_error	54 (22.9%)	529 (99.8%)
name_error	3 (1.3%)	1 (0.2%)
type_error	5 (2.1%)	0 (0.0%)
wrong_output	5 (2.1%)	0 (0.0%)
value_error	2 (0.8%)	0 (0.0%)
index_error	1 (0.4%)	0 (0.0%)
recursion_error	1 (0.4%)	0 (0.0%)
unknown	1 (0.4%)	0 (0.0%)

preference-based alignment. The confounds enumerated above prevent causal attribution.

6.0.4. 6.4 THE ZERO-REWARD BASIN HYPOTHESIS

The original hypothesis proposed that binary execution reward creates a "zero-reward basin" where all non-executing code receives identical zero reward, creating pressure to achieve syntactic validity first. The observed pattern—where CodeRL failures occur deeper in execution and include more assertion errors—is consistent with this hypothesis but does not confirm it, given the confounds.

An alternative explanation is that CodeT5’s encoder-decoder architecture and code-focused pre-training simply produce more syntactically valid code than CodeLlama’s decoder-only architecture with general-text pre-training, independent of RL training.

6.0.5. 6.5 EFFECT AMPLIFICATION AT FINE-GRAINED TAXONOMY

The increase in effect size from $V = 0.21$ to $V = 0.82$ at fine-grained taxonomy occurred because CodeLlama-Instruct errors concentrated almost exclusively in `syntax_error` (99.8%), while CodeRL errors distributed primarily between `indentation_error` (69.5%) and `syntax_error` (22.9%). This concentration vs. distribution pattern drives the larger chi-square statistic at the fine-grained level.

7. Conclusion

This study compared error type distributions between CodeRL (770M, execution-based RL) and CodeLlama-7B-Instruct (7B, instruction-tuned) on HumanEval+ and MBPP+. The models produced statistically dif-

ferent error distributions ($p < 10^{-7}$), with CodeRL showing more assertion errors (2.12% vs. 0%), deeper execution before failure (29.4% vs. 0.09% of lines), and more distributed fine-grained error causes.

These findings describe differences between two specific models but do not establish that alignment method is the causal factor. The models differ in architecture, size, pre-training data, and fine-tuning procedure. Controlled experiments training RL and DPO variants from identical base models would be required to isolate the effect of alignment method.

Future work should:

1. Compare RL and DPO variants trained from the same base model
2. Test models trained with code-specialized DPO rather than general instruction tuning
3. Evaluate across multiple model families and sizes
4. Extend to additional programming languages beyond Python

References

- Chen, M., Tworek, J., Jun, H., Yuan, Q., Pinto, H. P. d. O., Kaplan, J., Edwards, H., Burda, Y., Joseph, N., Brockman, G., et al. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021.
- Chen, Z., Li, Y., Wang, J., and Liu, Q. CodeDPO: Aligning code models with self-generated preferences for correctness and efficiency. *arXiv preprint arXiv:2502.01715*, 2025.
- Dou, S., Liu, Y., Jia, H., Wang, L., Zhou, E., Shen, W., Rao, J., Huang, Y., Xiong, F., et al. StepCoder: Improve code generation with reinforcement learning from compiler feedback. *arXiv preprint arXiv:2402.01391*, 2024.
- Le, H., Wang, Y., Gotmare, A. D., Savarese, S., and Hoi, S. C. CodeRL: Mastering code generation through pretrained models and deep reinforcement learning. In *Advances in Neural Information Processing Systems*, volume 35, pp. 21314–21328, 2022.

-
- Liu, J., Xia, C. S., Wang, Y., and Zhang, L. Is your code generated by ChatGPT really correct? rigorous evaluation of large language models for code generation. In *Advances in Neural Information Processing Systems*, volume 36, 2024a.
- Liu, X., Tian, R., Jia, Z., Bai, Y., Li, S., Li, Y., Li, D., Dou, S., Xiong, F., et al. RLEF: Grounding code LLMs in execution feedback with reinforcement learning. *arXiv preprint arXiv:2410.02089*, 2024b.
- Martin-Wey, L., Bouzenia, I., Fonseca, ., Delgado-Pérez, P., Médini, L., and Gallinari, P. CodeUltra-Feedback: An LLM-as-a-judge dataset for aligning large language models to coding preferences. *arXiv preprint arXiv:2403.09032*, 2024.
- Ouyang, L., Wu, J., Jiang, X., Almeida, D., Wainwright, C., Mishkin, P., Zhang, C., Agarwal, S., Slama, K., Ray, A., et al. Training language models to follow instructions with human feedback. In *Advances in Neural Information Processing Systems*, volume 35, pp. 27730–27744, 2022.
- Rafailov, R., Sharma, A., Mitchell, E., Ermon, S., Manning, C. D., and Finn, C. Direct preference optimization: Your language model is secretly a reward model. In *Advances in Neural Information Processing Systems*, volume 36, 2023.
- Rozière, B., Gehring, J., Gloeckle, F., Sootla, S., Gat, I., Tan, X., Adi, Y., Liu, J., Remez, T., Rapin, J., et al. Code Llama: Open foundation models for code. *arXiv preprint arXiv:2308.12950*, 2023.
- Shojaee, P., Jain, A., Tipirneni, S., and Reddy, C. K. PPOCoder: Execution-based code generation using deep reinforcement learning. *arXiv preprint arXiv:2305.06161*, 2023.
- Song, X. et al. MAPS: A noise-robust progressive learning approach for source-free domain adaptive keypoint detection. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2023.
- Touvron, H., Martin, L., Stone, K., Albert, P., Almahairi, A., Babaei, Y., Bashlykov, N., Batra, S., Bhargava, P., Bhosale, S., et al. Llama 2: Open foundation and fine-tuned chat models. *arXiv preprint arXiv:2307.09288*, 2023.
- Wang, X., Zhang, L., and Li, H. Focused-DPO: Enhancing code generation in error-prone regions. *arXiv preprint arXiv:2403.07974*, 2024.
- Wang, Y., Wang, W., Joty, S., and Hoi, S. C. CodeT5: Identifier-aware unified pre-trained encoder-decoder models for code understanding and generation. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*, pp. 8696–8708, 2021.
- Wang, Y., Chen, B., Zhang, J., and Liu, X. Understanding code generation failures: A systematic analysis of LLM-generated code errors. In *Proceedings of the 47th International Conference on Software Engineering*, 2025.
- Xu, S., Fu, W., Gao, J., Ye, W., Liu, W., Mei, Z., Wang, G., Yu, C., and Wu, Y. Is DPO superior to PPO for LLM alignment? a comprehensive study. *arXiv preprint arXiv:2404.10719*, 2024.
- Zhang, W., Li, M., Wang, T., and Chen, H. LlmFix: A large language model-based code repair framework with fine-grained error taxonomy. *arXiv preprint arXiv:2409.00676*, 2024.